

COMP 532

Machine Learning and Bioinspired Optimisation

Lecture 9 Reinforcement Learning



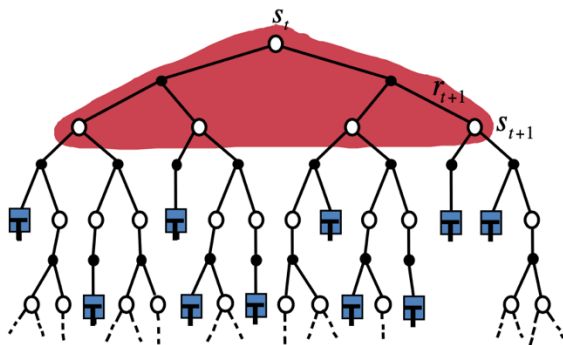
Meng Fang
University of Liverpool

Recap

- Reinforcement Learning
 - Solve the Bellman Optimality Equation

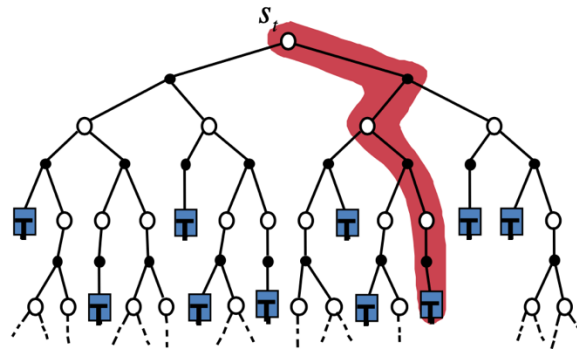
Recap

$$V(s) \leftarrow \sum_a \pi(a | s) \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V(s')]$$



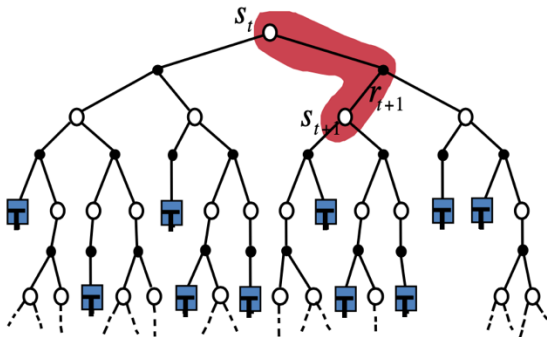
Dynamic Programming

$$V(s_t) \leftarrow V(s_t) + \alpha [R_t - V(s_t)]$$



Monte Carlo

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$



Temporal Difference

A Unifying Perspective

All these methods try to solve:

- $V = \textit{Bellman update}(V)$

DP:

- Use full expectation.

MC:

- Use full sample return.

TD:

- Use one-step sample + bootstrap.

Overview

- Problem Solving from Nature
- Reinforcement Learning (RL)
 - Roots of RL
 - Key features of RL
 - Elements of RL
 - Multi-armed bandits
 - Solve the problem in Markov Decision Processes (MDPs)
 - Value function
 - Solve the Bellman Optimality Equation
 - Temporal Difference (TD) Learning
- Deep Learning
- Multi-Agent Learning
- Swarm Intelligence

Temporal Difference Learning

- Introduce Temporal Difference (TD) learning
- Focus first on policy evaluation, or prediction, methods
- Then extend to control methods

TD Prediction

- Policy Evaluation (the prediction problem):
for a given policy π , compute the state-value function V^π

Simple every-visit Monte Carlo method:

$$V(s_t) \leftarrow V(s_t) + \alpha[R_t - V(s_t)]$$


target: the actual return after time t

The simplest TD method, TD(0):

$$V(s_t) \leftarrow V(s_t) + \alpha[\underbrace{r_{t+1} + \gamma V(s_{t+1})}_{\text{target}} - V(s_t)]$$


target: an estimate of the return
(TD error)

TD methods bootstrap and sample

- Bootstrapping: update estimates on the basis of other estimates
 - MC does not bootstrap
 - DP bootstraps
 - TD bootstraps
- Sampling: update does not involve an expected value
 - MC samples
 - DP does not sample
 - TD samples

Advantages of TD Learning

- TD methods do not require a model of the environment, only experience
- TD, but not MC, methods can be fully incremental
 - You can learn before knowing the final outcome
 - Less memory
 - Less peak computation
 - You can learn without the final outcome
 - From incomplete sequences

Classification of RL methods

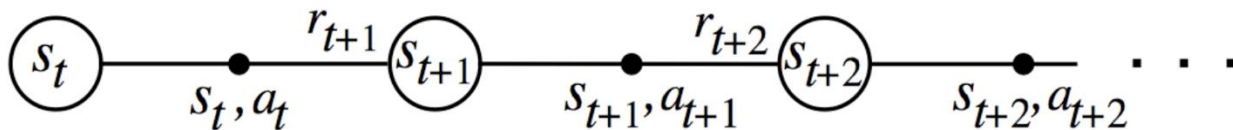
Model of the environment?

Bootstrap?

	YES	NO
YES	Dynamic programming	Temporal Difference (TD)
NO		Monte Carlo Methods

Learning an action-state value function

- Estimate Q^π for the current behaviour policy π .



- After every transition from a non-terminal state do this:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

Saras: on-policy TD control

- Turn this into a control method by always updating the policy to be greedy with respect to the current estimate:

Initialize $Q(s, a)$ arbitrarily

Repeat (for each episode):

Initialize s

Choose a from s using policy derived from Q (e.g., ϵ -greedy)

Repeat (for each step of episode):

Take action a , observe r, s'

Choose a' from s' using policy derived from Q (e.g., ϵ -greedy)

$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$

$s \leftarrow s', a \leftarrow a'$

until s is terminal

The current state at timestep t
The next state at timestep $t+1$

SARSA: State Action Reward State Action

Q-learning: off-policy TD control

- One-step Q-learning

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

The current state at timestep t

The next state at timestep $t+1$

Initialize $Q(s, a)$ arbitrarily

Repeat (for each episode):

Initialize s

Repeat (for each step of episode):

Choose a from s using policy derived from Q (e.g., ϵ -greedy)

Take action a , observe r, s'

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

$s \leftarrow s'$;

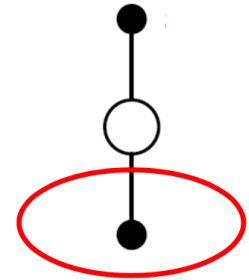
until s is terminal

Why off-policy?

On-policy vs. Off-policy

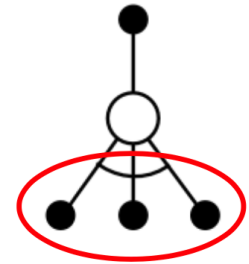
Sarsa:

Need to know the next action



$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

Q-learning:



$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Convergence to Optimality?

- Sarsa: difficult, but..
 - basic Sarsa converges given some assumptions
 - infinite visits to all state-action pairs
 - policy approaches greedy policy in the limit
- Q-learning: easier, because off-policy
 - converges given some assumptions
 - infinite visits..
 - decreasing step-size α (but not too fast)
- Even without these assumptions, but algorithms have shown good performance in many tasks!

Summary

- TD prediction
- Introduced one-step tabular model-free TD methods
- Extend prediction to control by employing some form of generalized policy iteration
 - On-policy control: Sarsa
 - Off-policy control: Q-learning
- These methods bootstrap and sample, combining aspects of DP and MC methods